

Contents

一、项目概述 1

二、数据挖掘 (Data Mining) 3

三、数据清洗 (Data Cleaning) 5

四、自然语言处理 (NLP) 7

五、信号构建与检验 (Signal) 9

六、回测 (Backtesting) 13

七、局限与改进 20

一、项目概述

围绕科技 / AI / 加密等领域 119 位 KOL 的推文，搭建完整流水线：

数据挖掘 → 数据清洗 → NLP → 信号 → 回测

模块	脚本	核心输出
数据挖掘	data_mining.py	users.csv 行业标注、 history_tweets.csv、kol_master.csv
数据清洗	data_cleaning.py	history_tweets_clean.csv (2343 行)、 tweet_events_clean.csv (1103 事件)
NLP	nlp.py	sentiment_score_nlp
信号	signal_testing.py	IC 扫描、signal_best_*.csv、多频率矩阵
回测	backtesting.py	净值、夏普、benchmark、多频率回测

标签：各行业代理 ETF 的次日收益 ind_ret_1d；交易为 lag1 (T 日信号 → T+1 收益)，避免同日泄露。

主结论速览 (最新跑批，lag1，成本 5bp)：

指标	全行业等权组合	分行业腿累加 (参考)
主信号	sig_kol_breadth_contrarian	同上
组合/腿日数	51 组合日	89 leg-days
总收益	+6.22%	+13.96%
夏普比率	2.55	2.51
最大回撤	-4.24%	-11.08%
IC (lag1)	—	0.255 (bootstrap p=0.007)

子样本回测 (探索，样本更短)：

范围	信号	交易日	总收益	夏普	最大回撤	IC
AI / 大模型	sig_kol_breadth_contrarian	30	+5.81%	3.08	-4.51%	0.40

范围	信号	交易日	总收益	夏普	最大回撤	IC
加密	sig_bear_cross	23d_strict	+6.63%	8.23*	-1.09%	0.46

* 加密仅 23 个交易日，夏普/年化易被放大，答辩作异质性补充，不作主结论。

二、数据挖掘 (Data Mining)

脚本: data_mining.py

输入: users.csv (119 位 KOL: 昵称、主页、阅读量、最近推文)

输出: 行业分类、Twikit 历史推文、kol_master.csv

2.1 流程

1. 行业分类: 根据主页/推文关键词将 KOL 划入 ai_tech、crypto、ai_tools 等, 并写回 users.csv。
2. 推文抓取: --fetch-twikit 按 KOL 拉取历史推文 (支持 --resume、--refetch-users)。
3. 主表构建: --master / --from-clean 生成 kol_master.csv、影响力排名 kol_ranked.csv。

2.2 行业一标的映射

行业代码	标签	代理 ETF
ai_tech	AI / 大模型	QQQ
ai_tools	AI 工具 / Agent	QQQ
crypto	加密	BTC-USD
semiconductor	半导体	SOXX
consumer_tech	消费科技	QQQ

2.3 规模 (当前跑批)

项目	数量
KOL 总数	119
原始推文 history_tweets.csv	约 2300+ 行
抓取窗口	近 90 天 (可增厚)

2.4 完整代码流程

对应 data_mining.py: 读 KOL 列表 → 规则分类 → 写回 users.csv → Twikit 抓推文 → 从清洗表建 kol_master。

```
import pandas as pd
from pathlib import Path
from data_mining import (
    read_users,
    rank_kols_by_industry,
    write_industry_to_users,
    export_rankings,
    fetch_posts,
    save_history,
    build_master_from_clean,
```

```

    USERS_CSV,
    HISTORY_CSV,
)

# ----- 1. 读 users.csv, 按行业排名 -----
users = read_users() # 从「首页地址」解析 username
ranked, top5 = rank_kols_by_industry(users, top_k=5)
# ranked: industry, views_seed, rank_in_industry, influence_score
write_industry_to_users(ranked) # 写回 industry / rank 列
export_rankings(ranked, top5) # -> data/outputs/kol_ranked.csv

# ----- 2. Twikit 拉历史推文 (需 cookies) -----
usernames = ranked["username"].tolist()
new_tweets = fetch_posts(
    usernames,
    days=90,
    max_per_user=120,
    sleep_user=10,
    resume=True, # 跳过 history 里已有用户
    retry_failed=True,
)
save_history(new_tweets, replace=False) # 追加到 history_tweets.csv

# ----- 3. 清洗完成后, 从 clean 表建主数据 -----
# 需先跑 data_cleaning.py + nlp.py
master = build_master_from_clean() # 合并 NLP、收益、行业标签
master.to_csv("data/outputs/kol_master.csv", index=False, encoding="utf-8-sig")
print(f"kol_master: {len(master)} 行")

行业分类核心逻辑 (classify_profile + rank_kols_by_industry):
def classify_profile(nickname, username, latest: str) -> str:
    blob = f"{nickname} {username} {latest}".lower()
    for kw in _INDUSTRY_PATTERNS: # ai, llm, crypto, btc, ...
        if kw in blob:
            return INDUSTRY_MAP[kw] # -> ai_tech / crypto / ...
    return "unclassified"

ranked = ranked.sort_values(["industry", "views_seed"], ascending=[True, False])
ranked["rank_in_industry"] = ranked.groupby("industry").cumcount() + 1
ranked["influence_score"] = ranked.groupby("industry")["views_seed"].transform(
    lambda s: round(100.0 * s / s.max(), 2) if s.max() else 0.0
)

```

三、数据清洗 (Data Cleaning)

脚本: data_cleaning.py

输入: history_tweets.csv

输出: history_tweets_clean.csv、tweet_events_clean.csv

3.1 流程

1. 文本规范化: 去 URL/多余空白, 保留 text_raw。
2. 质量过滤: 低互动、疑似机器人 (freq / likes_low 等)。
3. 标的解析: cashtag + 行业 → ticker / industry_ticker; 黑名单符号 (如 K、POD) 回退行业 ETF。
4. 时间修复: repair_tweet_times() 从 tweet_id 恢复 UTC 时间 (Snowflake)。
5. 收益对齐: yfinance 拉日线/5 分钟价, 计算 ind_ret_1d 等。

3.2 清洗结果

指标	数值
清洗后推文	2343 行
有效事件 (去重后)	1103 条
含有效 ind_ret_1d	543 条 (约 49%; 其余 pending / 非交易日)
history 覆盖 KOL	104 / 119 (可继续抓取增厚)

3.3 完整代码流程

对应 data_cleaning.py 的 run_cleaning(): 加载原始推文 → 清洗标注 → 词典情感 (粗) → yfinance 对齐收益 → 导出事件子集。

```
import pandas as pd
import numpy as np
from data_cleaning import (
    load_history,
    repair_tweet_times,
    annotate_cleaning,
    _apply_lexicon_sentiment,
    aligned_returns,
    select_event_rows,
    enrich_returns_only,
    HISTORY_CLEAN_CSV,
    EVENTS_CLEAN_CSV,
)

# ----- 全量清洗 (等价于 python3 data_cleaning.py) -----
def run_cleaning_pipeline(skip_returns: bool = False):
    df = load_history()                # history_tweets.csv
    df = repair_tweet_times(df)        # Snowflake 修 tweet_time
```

```

df = annotate_cleaning(df) # 文本/行业/ticker/机器人标记
df = _apply_lexicon_sentiment(df) # sentiment_score (词典, NLP 会覆盖)
if not skip_returns:
    df = aligned_returns(df) # yfinance -> ind_ret_1d 等
df.to_csv(HISTORY_CLEAN_CSV, index=False, encoding="utf-8-sig")
events = select_event_rows(df, drop_bots=True)
events.to_csv(EVENTS_CLEAN_CSV, index=False, encoding="utf-8-sig")
return df, events

df, events = run_cleaning_pipeline()

# ----- 仅刷新收益 (等价于 --returns-only) -----
# enriched = enrich_returns_only() # 内部: repair -> _refresh_tickers -> aligned_returns

时间修复 (repair_tweet_times):
# 优先级: CSV 已有时间 -> snowflake(tweet_id) -> ingested_at
sf = (int(tweet_id) >> 22) + 1288834974657 # ms since epoch
tweet_time = pd.to_datetime(sf, unit="ms", utc=True)

标的解析 + 收益对齐要点:
# annotate_cleaning 内: 正文 cashtag + 行业 -> ticker / industry_ticker
ticker, ticker_src, ind_ticker = _resolve_tickers(text_raw, industry)
# 黑名单 K/POD 等 -> 回退行业 ETF, 避免 yfinance 报错

# aligned_returns: 批量下载日线, 事件日收盘价 -> 下一交易日 ind_ret_1d
daily_cache, intra_cache = _price_batch_cache(df)
# 对每行: anchor 日 close, forward 1d/5d/20d; crypto 用 UTC 日历

事件筛选 (select_event_rows): 去掉 is_bot、无行业、无有效时间等行, 供信号与回测使用。

```

四、自然语言处理 (NLP)

脚本: nlp.py

输入: history_tweets_clean.csv

输出: 帖级 sentiment_score_nlp (写入 clean / events)

4.1 方法

- 对推文正文做情感打分, 范围约 $[-1, 1]$ (负 = 看空, 正 = 看多)。
- 信号层使用 `-sentiment_mean` 构成反转因子 (情绪越偏空 → 信号越强 → 博弈次日反弹)。

4.2 在因子中的角色

日频聚合 (按 event_date × industry):

- sentiment_mean: 当日情感均值
- bear_posts / bull_posts: 看空/看多帖计数
- sentiment_dispersion: 分歧度 (用于 sig_unanimity_contrarian 等)

4.3 完整代码流程

对应 nlp.py 的 run_nlp(): 读清洗表 → 多源情感融合 → 写回 sentiment_score_nlp → 同步事件表。

```
import pandas as pd
from data_cleaning import repair_tweet_times, select_event_rows, HISTORY_CLEAN_CSV, EVENTS_CLEAN_CSV
from nlp import enrich_dataframe, composite_scores, lexicon_score, snownlp_score

# ----- 1. 加载并修复时间 -----
df = repair_tweet_times(pd.read_csv(HISTORY_CLEAN_CSV))
text_col = "text_clean" if "text_clean" in df.columns else "text"
texts = df[text_col].fillna("").astype(str).tolist()

# ----- 2. 单条推文: 词典 + SnowNLP 加权融合 -----
# composite_scores 对每条文本:
#   lexicon_score (中英看空/看多词表, 权重 0.45)
#   snownlp_score (可选, 权重 0.25)
#   transformer (可选 --transformers, 权重 0.30)
labels, scores = composite_scores(texts, use_transformers=False)

df = df.copy()
df["sentiment_nlp"] = labels
df["sentiment_score_nlp"] = scores # [-1, 1], 信号里用 -sentiment_mean

# ----- 3. 写回 clean + events -----
df.to_csv(HISTORY_CLEAN_CSV, index=False, encoding="utf-8-sig")
```

```
events = select_event_rows(df, drop_bots=True)
events.to_csv(EVENTS_CLEAN_CSV, index=False, encoding="utf-8-sig")
print(f"NLP done: {len(df)} tweets, {len(events)} events")
```

```
# 等价一行: enrich_dataframe(df, use_transformers=False)
```

词典打分示例 (lexicon_score):

```
def lexicon_score(text: str) -> tuple[str, float]:
    t = clean_urls_and_cashtags(text).lower()
    p = sum(1 for w in POS_ZH if w in t) # 利好词计数
    n = sum(1 for w in NEG_ZH if w in t) # 利空词计数
    score = (p - n) / max(p + n, 1)      # 归一化到 [-1, 1]
    return label_from_score(score), score
```

五、信号构建与检验 (Signal)

脚本: signal_testing.py

输入: tweet_events_clean.csv (含 NLP 与收益)

输出: signal_leaderboard.csv、signal_best_all.csv、signal_best_per_industry.csv

5.1 信号释义

流程: 帖级打分 → 按 event_date × industry 聚合 → 日频衍生因子。下表先说明「是什么」, 再单独给出「怎么算」。

表 1: 信号名与含义 (经济直觉)

信号名	含义
sig_kol_breadth_contrarian	讨论该行业的 KOL 越多、情绪 越偏空, 次日行业 ETF 越容易反弹 (拥挤恐慌反转)
sig_bear_crowd	看空帖占比高, 且整体情绪偏空 → 赌恐慌后的反弹
sig_contrarian_top3	只听取行业内 排名前 3 的 KOL, 对其情绪 反向 交易
sig_dispersion_contrarian	观点 分歧大 且整体偏空时, 反转信号 更强
sig_unanimity_contrarian	观点 高度一致 (低分歧) 且偏空时, 一致预期更容易被 打脸
sig_bear_crowd_strict	仅当 超过半数帖子看空 时才触发恐慌反转, 过滤零星看空噪音
sig_rank1_momentum	跟随 行业影响力 #1 KOL 的情绪方向 (动量, 与主反转逻辑相反)
sig_contrarian (基础)	单帖层面的 情绪反转 × 影响力, 是多数日频反转信号的构建块

表 2: 信号名与计算方式 (日频)

信号名	计算方式
sig_kol_breadth_contrarian	$n_kol * (-sentiment_mean) * influence_sum / n_posts$
sig_bear_crowd	$bear_ratio * (-sentiment_mean) * influence_sum$, 其中 $bear_ratio = bear_posts / n_posts$
sig_contrarian_top3	帖级: 仅 $kol_rank_in_industry \leq 3$ 时 $(-sentiment) * influence$; 日频: sum
sig_dispersion_contrarian	$(-sentiment_mean) * sentiment_dispersion * influence_sum$
sig_unanimity_contrarian	$(-sentiment_mean) * influence_sum / (sentiment_dispersion + 0.15)$

信号名	计算方式
sig_bear_crowd_strict	若 bear_ratio > 0.5: bear_ratio * (-sentiment_mean) * influence_sum, 否则 0
sig_rank1_momentum	帖级: 仅 kol_rank_in_industry == 1 时 sentiment * influence; 日频: sum
sig_contrarian (基础)	帖级: (-sentiment) * influence; 日频可对帖级列 sum 得到日度反转强度

符号说明: sentiment_mean = 当日行业情感均值; influence_sum = 当日影响力之和; sentiment_dispersion = 当日帖间情感标准差; n_kol / n_posts = 参与 KOL 数 / 帖数。

反转 vs 动量: 名称含 contrarian / crowd 的多半预测 次日反向; momentum 预测 次日同向。

5.2 信号构建代码 (完整流程)

对应 signal_testing.py: load_events → build_signal_variants(帖级) → aggregate_daily_signals (日频) → enrich_daily_signals (日频衍生)。

```
import numpy as np
import pandas as pd
from data_cleaning import repair_tweet_times

# ----- 1. 加载事件表并修复时间 -----
events = repair_tweet_times(pd.read_csv("data/outputs/tweet_events_clean.csv"))
events["tweet_time"] = pd.to_datetime(events["tweet_time"], utc=True)
events["event_date"] = events["tweet_time"].dt.date

# 合并 KOL 影响力 (users.csv / kol_ranked.csv)
ranked = pd.read_csv("data/outputs/kol_ranked.csv")[["username", "influence_score", "kol_rank_in_industry"]]
work = events.merge(ranked, on="username", how="left")
work["influence"] = work["influence_score"].fillna(work.get("views_seed", 1)).clip(lower=1)
work["sentiment"] = pd.to_numeric(work["sentiment_score_nlp"], errors="coerce").fillna(0)
rank = pd.to_numeric(work["kol_rank_in_industry"], errors="coerce").fillna(5)

# ----- 2. 帖级信号 (每条推文一个标量) -----
s, inf = work["sentiment"], work["influence"]
work["sig_contrarian"] = (-s) * inf
work["sig_contrarian_top3"] = np.where(rank <= 3, (-s) * inf, 0.0)
work["sig_rank1_momentum"] = np.where(rank <= 1, s * inf, 0.0)

# ----- 3. 日频聚合: event_date × industry -----
gcols = ["event_date", "industry", "industry_label"]
signal_cols = ["sig_contrarian", "sig_contrarian_top3", "sig_rank1_momentum"]

daily = work.groupby(gcols, as_index=False).agg(
    n_posts=("tweet_id", "count"),
    n_kol=("username", "nunique"),
```

```

influence_sum=("influence", "sum"),
sentiment_mean=("sentiment", "mean"),
bear_posts=("sentiment", lambda x: int((x < -0.1).sum())),
bull_posts=("sentiment", lambda x: int((x > 0.1).sum())),
ind_ret_id=("ind_ret_id", "mean"),
**{f"{c}_sum": (c, "sum") for c in signal_cols},
)
disp = work.groupby(gcols)["sentiment"].std().rename("sentiment_dispersion")
daily = daily.merge(disp.reset_index(), on=gcols, how="left")

# ----- 4. 日频衍生信号（行业面板上的公式列） -----
sm = daily["sentiment_mean"].fillna(0)
inf = daily["influence_sum"].fillna(1.0)
n = daily["n_posts"].clip(lower=1)
bear_r = daily["bear_posts"] / n
disp = daily["sentiment_dispersion"].fillna(0)

daily["sig_kol_breadth_contrarian_sum"] = (
    daily["n_kol"].fillna(0) * (-sm) * inf / n
)
daily["sig_bear_crowd_sum"] = bear_r * (-sm) * inf
daily["sig_bear_crowd_strict_sum"] = np.where(bear_r > 0.5, bear_r * (-sm) * inf, 0.0)
daily["sig_dispersion_contrarian_sum"] = (-sm) * disp * inf
daily["sig_unanimity_contrarian_sum"] = (-sm) * inf / (disp + 0.15)
# sig_contrarian_top3_sum / sig_rank1_momentum_sum 已在步骤 3 聚合完成

# ----- 5. IC 检验 (lag1: T 日信号 vs T+1 收益) -----
panel = daily.dropna(subset=["ind_ret_id"]).sort_values(["industry", "event_date"])
panel["signal_lag1"] = panel.groupby("industry")["sig_kol_breadth_contrarian_sum"].shift(1)
ic = panel[["signal_lag1", "ind_ret_id"]].corr(method="spearman").iloc[0, 1]
print(f"sig_kol_breadth_contrarian IC(lag1) = {ic:.4f}")

```

5.3 信号选用（主信号与分行业探索）

表 1: 全行业通用信号（统一公式，用于主回测）

角色	信号名	IC (lag1)	有效腿数	用途
主信号	sig_kol_breadth_contrarian	0.255	84	主回测、答辩主结论
备选	sig_bear_crowd	0.241	84	强调看空帖占比
备选	sig_contrarian_top3	0.240	84	Top3 KOL 反转

以上信号在 所有行业 使用同一日频公式，不随行业切换。

表 2：分行业探索信号（各行业单独 IC 选优，样本较短）

行业	信号名	IC (lag1)	有效腿数	含义
AI / 大模型 (ai_tech)	sig_unanimity_contrarian	0.439	38	低分歧时加强反转
加密 (crypto)	sig_bear_crowd_strict	0.458	22	看空帖占比 >50% 才触发
AI 工具 (ai_tools)	sig_rank1_momentum	0.605	22	仅 #1 KOL 动量（与主逻辑相反）

分行业信号 不替代主信号；consumer_tech、semiconductor 腿数过少，未列入上表。

选用原则：

- 生产 / 主故事：全行业统一 sig_kol_breadth_contrarian (IC 0.255, bootstrap **p=0.007**)。
- 子样本验证（已回测）：ai_tech 用同一主信号仍有效 (IC 0.40) ；crypto 可试 sig_bear_crowd_strict (IC 0.46, 23 日)。
- 不写主结论：ai_tools（全行业回测亏损）、consumer_tech / semiconductor（腿数 <=2）。

5.4 IC 扫描（全行业 Top, lag1）

信号	IC	n_days	说明
sig_kol_breadth_contrarian	0.255	84	主信号
sig_bear_crowd	0.241	84	恐慌占比
sig_contrarian_top3	0.240	84	Top3
sig_dispersion_contrarian	0.208	84	高分歧反转

5.5 稳健性（诊断）

指标	全样本	训练集 (70%)	测试集 (30%)
IC (lag1)	0.255	0.096	0.314
自助法 p 值（双侧）	0.007	—	—
IC 95% CI	[0.081, 0.441]	—	—

六、回测 (Backtesting)

脚本: backtesting.py
主信号: sig_kol_breadth_contrarian
模式: lag1 | 成本: 单边 5bp | 收益列: ind_ret_1d

6.1 全样本绩效 (核心表)

等权组合 = 每个交易日, 对有交易的行业腿取平均 PnL, 再复利成一条净值 (对外汇报用此口径)。

指标	等权组合 (full)	等权组合 (test)	分行业腿累加 (full)
交易/组合日数	51	16	89 leg-days
总收益	+6.22%	+2.13%	+13.96%
年化收益	+34.8%	+39.3%	+44.8%
年化波动	13.7%	14.6%	17.9%
夏普比率	2.55	2.69	2.51
最大回撤	-4.24%	-3.01%	-11.08%
胜率	52.9%	56.3%	51.7%
IC (lag1)	—	—	0.255
日均 PnL	+0.122%	+0.136%	+0.153%

训练 / 测试切分 (分行业腿口径, 70% / 30%)

切分	总收益	年化收益	夏普	最大回撤	IC
训练 (46 腿)	+4.94%	+30.3%	2.01	-4.32%	0.102
测试 (43 腿)	+8.59%	+62.1%	3.01	-7.07%	0.314

敏感性 (诊断): 去掉 crypto 后, 全行业等权组合收益约 +0.99% (alpha 部分依赖加密); 全样本诊断组合收益约 +6.50% (含成本口径差异)。

6.2 全行业回测下的分行业分解 (统一主信号)

全行业命令: python3 backtesting.py --all --signal sig_kol_breadth_contrarian --mode lag1

行业	腿数	总收益	夏普	最大回撤	IC	胜率	备注
AI / 大模型	39	+5.81%	3.08	-4.51%	0.40	56.4%	主贡献行业
加密	23	+11.26%	11.73*	-2.63%	0.29	60.9%	同一主信号
AI 工具 / Agent	23	-8.27%	-3.70	-9.55%	-0.01	34.8%	建议剔除

行业	腿数	总收益	夏普	最大回撤	IC	胜率	备注
消费科技 / 半导体	2	—	—	—	—	—	样本过少

* 加密在全行业表中年化/夏普因样本短被放大，仅作参考。

6.3 分行业专用信号回测（最新验证）

在子行业内使用 IC 扫描推荐信号（样本短，作探索）：

范围	命令要点	信号	交易日	总收益	夏普	最大回撤	IC
ai_tech	--industry ai_tech	sig_kol_breadth_contrarian	2019-01-01 ~ 2020-01-01	+5.81%	3.08	-4.51%	0.396
crypto	--industry crypto	sig_bear_crowd_strict	2019-01-01 ~ 2020-01-01	+6.63%	8.23*	-1.09%	0.458

```
python3 backtesting.py --industry ai_tech --signal sig_kol_breadth_contrarian --mode lag1
python3 backtesting.py --industry crypto --signal sig_bear_crowd_strict --mode lag1
```

注意：ai_tech 测试集（12 日）夏普约 -0.57，说明一切分样本波动很大；crypto 测试集仅 7 日，统计意义有限。答辩主数字仍用第六章 6.1 全行业等权组合。

6.4 交易策略与方法（信号如何变成交易）

本节说明回测里「日频信号 → 仓位 → 收益 → 组合净值」的完整规则；后文回撤均在该规则下产生。

6.4.1 交易标的与粒度

项目	规则
标的	每条推文所属行业的 代理 ETF（如 ai_tech→QQQ, crypto→BTC-USD）
决策粒度	按 event_date × industry 聚合为「行业日」；同一行业当日多条推文合成一个信号
收益标签	ind_ret_1d = 事件日收盘 → 下一交易日收盘的行业 ETF 收益率（已在清洗阶段对齐）

6.4.2 从信号到仓位（lag1，主回测口径） Step 1 一日频信号（以主信号为例）：

```
signal_raw(T) = sig_kol_breadth_contrarian_sum
                = n_kol(T) * (-sentiment_mean(T)) * influence_sum(T) / n_posts(T)
```

信号 越大 → 当日 KOL 讨论越广且情绪越偏空 → 博弈 次日反弹。

Step 2 一滞后一天使用信号（避免未来函数）：

signal_lag1(T) = signal_raw(T-1) # 每个 industry 组内 shift(1)

Step 3 一多空方向（方向性做多空，非中性化）：

position_lag1(T) = sign(signal_lag1(T)) # +1 做多 / -1 做空 / 0 空仓

signal_lag1	含义（反转逻辑）	仓位	持有区间
> 0	昨日偏「拥挤恐慌」	+1 做多 行业 ETF	交易日 T 收盘 → T+1 收盘
< 0	昨日偏「一致看多」	-1 做空 行业 ETF	同上
= 0	无明确方向	0 空仓	不交易

Step 4 一单腿盈亏与成本：

pnl_gross(T) = position_lag1(T) * ind_ret_1d(T)
pnl_net(T) = pnl_gross(T) - |Δposition| * (5bp / 10000) # 换仓扣单边 5bp

实现见 signal_testing.build_daily_panel 与 finalize_panel（分行业分别累计净值）。

6.4.3 组合构建（全行业 --all） 每个交易日 T：

- 1. 对所有 当日有信号且有 ind_ret_1d 的行业腿，各算一条 pnl_net；
- 2. 等权平均 得到组合当日收益：pnl_combo(T) = mean(pnl_net over industries)；
- 3. 组合净值：equity(T) = cumprod(1 + pnl_combo)。

不跨行业对冲、不做市值加权；每条腿独立多空行业 ETF，再在日频上平均。

6.4.4 时间线示例（与回撤对应）

T-1 日收盘后：根据 T-1 日 KOL 推文算 signal_raw(T-1)
T 日开盘前：确定 position_lag1(T) = sign(signal_raw(T-1))
T 日收盘 → T+1 收盘：实现 ind_ret_1d(T)，计入 pnl_lag1(T)

2026-05-07 ~ 05-09 回撤段（组合最深 -4.24%）：多日 多个行业同时做多或做空，但 ind_ret_1d 与 position 方向相反（或换仓成本叠加），等权组合连续三日负收益；05-15 单日多行业正向 pnl 推动净值收复前高。

6.4.5 与 contemp 模式的区别（仅作对照）

模式	信号使用	用途
lag1（主）	用 昨日 信号交易 今日 已实现收益	可交易、无同日泄露

模式	信号使用	用途
contemp	用 当日 信号 × 当日收益	仅看拟合上限，不用于答 辩主结论

```
# signal_testing.py —核心两行
daily["signal_lag1"] = daily.groupby("industry")["signal_raw"].shift(1)
daily["position_lag1"] = np.sign(daily["signal_lag1"]).fillna(0)
daily["pnl_lag1"] = daily["position_lag1"] * daily["ind_ret_1d"]
```

6.5 最大回撤与日期

以下回撤均在 6.4 节 lag1 + 等权组合 + 5bp 规则下计算（strategy_equity_combined.csv）。
组合净值

项目	数值
最大回撤	-4.24%
峰值日	2026-04-20（净值约 1.061）
谷底日	2026-05-09（净值约 1.016）
恢复至前高	2026-05-15

谷底附近最差交易日

日期	当日组合 PnL	相对前高回撤
2026-05-09	-0.80%	-4.24%（最深）
2026-05-08	-0.94%	-3.46%
2026-05-07	-1.30%	—
2026-05-03	-0.78%	-2.62%

早期回撤：2026-02-17 → 2026-02-23，约 -2.42%（仅 1 个行业有交易，参考价值较低）。
分行业腿回撤（峰值日 → 谷底日）

行业	最大回撤	峰值日	谷底日
AI / 大模型	-4.51%	2026-05-04	2026-05-10
加密	-2.63%	2026-05-06	2026-05-09
AI 工具 / Agent	-9.55%	2026-02-17	2026-05-14

6.6 完整代码流程

对应 backtesting.py 的 run_strategy()：事件表 → 帖级信号 → 日频面板 → lag1 仓位 → 扣费 → 等权组合净值 → 绩效统计。

```

import numpy as np
import pandas as pd
from signal_testing import (
    load_events,
    build_signal_variants,
    build_daily_panel,
    finalize_panel,
    build_combined_portfolio,
    performance_stats,
    performance_stats_combined,
    time_split,
    backtest_by_industry,
)
from backtesting import import filter_tweets

SIGNAL = "sig_kol_breadth_contrarian"
RET_COL = "ind_ret_1d"
COST_BPS = 5.0
MODE = "lag1"

# ----- 1. 加载事件 + 帖级信号 -----
raw = load_events()
tweets = build_signal_variants(raw)
tweets = filter_tweets(tweets, industry=None, max_rank=None) # 全行业

# ----- 2. 日频面板: T 日信号, fwd_ret = 当日行的 ind_ret_1d (已是 T+1 收益) -----
panel = build_daily_panel(tweets, SIGNAL, ret_col=RET_COL)
# build_daily_panel 内:
# aggregate_daily_signals -> enrich_daily_signals
# signal_lag1 = groupby(industry).shift(1) # 用昨日信号
# position_lag1 = sign(signal_lag1)
# pnl_lag1 = position_lag1 * fwd_ret

pnl_col = "pnl_lag1"
pos_col = "position_lag1"
panel = finalize_panel(panel, pnl_col, pos_col, COST_BPS, MODE, SIGNAL)
# finalize_panel: 分行业扣换手费 -> pnl_net; 分行业累计 equity_industry

# ----- 3. 等权组合净值 -----
combined = build_combined_portfolio(panel)
# 每个 event_date: 各行业 pnl_net 取平均 -> 一条组合曲线

# ----- 4. 绩效指标 (收益、夏普、回撤、IC) -----
train, test = time_split(panel, train_ratio=0.7)
stats_full = performance_stats(panel, pnl_col, pos_col, split="full", net_pnl=True)
stats_test = performance_stats(test, pnl_col, pos_col, split="test", net_pnl=True)
comb_stats = performance_stats_combined(combined, split="combined_full")
by_ind = backtest_by_industry(panel, pnl_col, pos_col)

```

```
print(" 组合总收益:", comb_stats["total_return"])
print(" 组合夏普:", comb_stats["sharpe"])
print(" 组合最大回撤:", comb_stats["max_drawdown"])
print(" 全样本 IC:", stats_full.get("ic_signal_fwd_ret"))
```

lag1 交易逻辑（避免未来函数）：

```
daily["signal_raw"] = daily["sig_kol_breadth_contrarian_sum"]
daily["signal_lag1"] = daily.groupby("industry")["signal_raw"].shift(1)
daily["position_lag1"] = np.sign(daily["signal_lag1"]).fillna(0)
daily["pnl_lag1"] = daily["position_lag1"] * daily["ind_ret_1d"] # 标签已是次日收益
net = pnl - turnover * (cost_bps / 10000) # 换仓扣 5bp
equity = (1 + net).cumprod()
```

6.7 输出文件

文件	内容
strategy_backtest_report_all.csv	收益、夏普、回撤、IC 汇总
strategy_equity_combined.csv	组合净值曲线
strategy_daily_trades_all.csv	日 x 行业交易明细
strategy_backtest_by_industry.csv	分行业绩效
signal_diagnostics_report.csv	bootstrap、ex-crypto

6.8 多频率 signal 扫描（频率 x 行业）

命令：

```
python3 signal_testing.py --scan-multi-horizon
```

设计：5m / 1h 用 lag=0 (contemp)；1d / 5d / 20d 用 lag=1。在每个（行业，频率）子样本上对全部 sig_* 算 IC，取 IC 最高者。

帖级标签覆盖：5m 638 条 | 1h 631 | 1d 543 | 5d 208 | 20d 46。

IC 最优矩阵（分行业）：

行业	5 分钟	1 小时	1 日	5 日	20 日
AI / 大模型	0.35	0.29	0.44 (unanimity)	0.49	0.65*
	(nlp_lex_disagree)				
加密	0.40	0.09	0.46 (bear_strict)	0.64	0.81*
	(rt_contrarian)				
AI 工具	0.73*	0.54*	0.61*	0.79*	0.50*
			(rank1_momentum)		

* 20d 等长周期标签极少（≤15 行业日），IC 仅作探索。

输出：signal_matrix_frequency_industry.csv、signal_leaderboard_multi_horizon.csv。

6.9 多频率回测（矩阵逐格验证）

命令：

```
python3 backtesting.py --backtest-multi-horizon
```

在 IC 矩阵基础上，对每个（行业，频率，最优 signal）跑 与扫描一致的 lag / ret_col / mode，验证 IC 能否转化为 PnL。

1 日频（主交易口径，lag1，5bp）：

行业	最优 signal	IC	腿收益	夏普	最大回撤
AI / 大模型	sig_unanimity_contrarian	0.44	+5.81%	3.08	-4.51%
加密	sig_bear_crowd_strict	0.46	+6.63%	8.23	-1.09%
AI 工具	sig_rank1_momentum	0.61	+8.65%	11.04	-1.61%

其他频率（探索，样本更短）：

行业	频率	signal	腿收益	夏普	备注
加密	5 日	sig_rt_contrarian	+17.0%	21.5	19 腿-days
AI 工具	5 日	sig_rank1_momentum	+32.6%	127*	19 腿-days, 夏普易失真
AI 大模型	1 小时	sig_strong_bull_fall	+4.54%	-3.14	IC 高但 PnL 为负
加密	5 分钟	sig_rt_contrarian	+0.03%	0.20	短频几乎无增量

结论：

- 1. 可交易主结论仍落在 1 日 lag1；ai_tech / crypto 分行业最优与 6.1—6.4 一致。
- 2. IC 高 ≠ 能赚钱（如 ai_tech 1h、5d：IC>0.29 但回测亏损）。
- 3. ai_tools 用 rank1_momentum（跟随 #1 KOL）优于全行业统一的 breadth 反转——说明行业路由必要。
- 4. 5m/1h/20d 受 lag=0 时序 或 样本过少 约束，不作主结论。

输出：signal_matrix_backtest.csv。

6.10 策略 Benchmark 对比（统一口径）

命令：

```
python3 backtesting.py --compare-benchmarks --all --ret-col ind_ret_1d --mode lag1
```

全行业、同一 ind_ret_1d、lag1、成本 5bp，对比基准与候选策略（等权组合指标为主）：

策略	组合总收益	组合夏普	最大回撤	腿 IC
买入持有 (有事件日 做多)	+7.49%	2.93	-4.75%	—
始终空仓	0%	—	0%	—
随机多空	-10.92%	-3.23	-12.25%	-0.24
跟随情绪 baseline	-10.20%	-2.98	-12.36%	-0.19
反转 contrarian	+6.65%	2.75	-4.20%	0.19
主信号 breadthx 反转	+6.22%	2.55	-4.24%	0.255
ML 二分类 sig_ml_up	-3.93%	-1.43	-5.99%	0.04
分行业 1d IC 路由	+11.75%	7.33*	-1.93%	0.263

* 分行业路由夏普在 51 组合日上易被放大，但 收益与回撤均优于单一主信号。

Benchmark 结论：

1. 主信号 **显著 beat** 随机、baseline（跟随情绪），与 contrarian 家族一致。
2. 主信号 **略逊于**「有 KOL 事件日无脑做多」的买入持有（+7.5% vs +6.2%），但 **IC 更高、逻辑可解释**；实盘中可叠加 **仅强信号开仓** 以控制 beta。
3. **ML 对照最弱**，支持「小样本 + 可解释规则」路线。
4. **分行业路由**（ai_tech→unanimity, crypto→bear_strict, ai_tools→rank1_momentum）为当前最优组合，后续产品化优先。

输出：strategy_benchmark_comparison.csv。

七、局限与改进

1. **样本偏短**：约 84 个 IC 腿、51 个组合日；多频率 20d 仅 46 帖级标签；需继续增厚。
2. **crypto 依赖**：剔除加密后组合收益约 +1%（见 6.10 买入持有仍含 crypto beta）。
3. **IC vs PnL 背离**：短频 / 5d 扫描 IC 高但回测可为负（6.9）；勿只看 IC 选 signal。

4. 分行业路由更优但待 OOS: 1d 路由组合 +11.75%, 夏普在短窗上可能高估; 样本 >100 日后再上线。
5. ML / 复杂模型: sig_ml_up benchmark 垫底, 暂不作主策略。